

Área 5 PMI: Gestión de la Calidad

Descripción del área

La Gestión de la Calidad es el área del PMBOK responsable de definir los estándares de calidad del proyecto y garantizar que los entregables los cumplan. En un proyecto de software, la calidad no es un paso final de verificación; es un proceso continuo que se construye durante cada sprint.

En este proyecto, la calidad se gestiona mediante dos contratos internos del equipo: la Definition of Ready (DoR), que define cuándo una historia de usuario puede entrar a un sprint, y la Definition of Done (DoD), que define cuándo una historia de usuario está realmente terminada. Ambas definiciones son acordadas por el equipo completo al inicio de la Fase 2 y se respetan durante todos los sprints.

Definition of Ready (DoR) — 5 criterios

Una historia de usuario puede ingresar al Sprint Planning únicamente si cumple los 5 criterios del DoR. Si no los cumple, el SM rechaza la HU y el PO debe refinarla antes del siguiente Sprint Planning.

#	Criterio	Descripción aplicada al proyecto
1	Historia redactada en formato estándar	La HU usa el formato: “Como [aspirante / administrador / sistema], quiero [funcionalidad], para [beneficio esperado].”
2	Criterios de aceptación definidos	Cada HU tiene al menos 3 criterios de aceptación concretos y verificables acordados con el PO.
3	Estimación en story points acordada	El equipo ha estimado la HU en Planning Poker y existe consenso. El rango aceptable es 1–8 SP por HU para este proyecto.

#	Criterio	Descripción aplicada al proyecto
4	Dependencias identificadas y resueltas	Si la HU depende de otra HU o de un componente externo (como el catálogo de Admisiones), la dependencia está documentada y no bloquea la implementación.
5	Sin ambigüedad técnica	El equipo técnico ha revisado la HU y no existe ambigüedad sobre la implementación. Si hay ambigüedad, se resuelve en el Refinement antes del Sprint Planning.

Definition of Done (DoD) — 7 criterios

Una historia de usuario se marca como “Done” únicamente cuando cumple los 7 criterios del DoD. El SM verifica el DoD antes de presentar la HU en el Sprint Review. Una HU que no cumple el DoD completo no puede mostrarse al cliente como funcionalidad entregada.

#	Criterio	Descripción aplicada al proyecto
1	Código desarrollado y en repositorio	El código fuente de la funcionalidad está commiteado en la rama correspondiente del repositorio del proyecto.
2	Tests unitarios escritos y pasando (TDD)	Los tests unitarios que cubren la lógica de la HU fueron escritos antes o durante el desarrollo (TDD) y están pasando en el entorno de CI.

#	Criterio	Descripción aplicada al proyecto
3	Code Review completado (Pair Programming / XP)	El código fue revisado por al menos un developer distinto al autor. La columna Code Review del tablero Kanban/Scrumban fue usada para este control.
4	Integración continua (CI) sin errores	El pipeline de CI ejecuta exitosamente: compilación, tests unitarios y análisis estático sin errores bloqueantes.
5	Pruebas funcionales pasando	Los criterios de aceptación definidos en el DoR han sido verificados manualmente o de forma automatizada. Todos los criterios pasan sin observaciones abiertas.
6	Revisión de seguridad básica	Para HU que manejen datos de aspirantes (EP01, EP02, EP06), el developer Chen ha revisado que no existan vulnerabilidades de exposición de datos personales (riesgo R8).
7	Documentación técnica mínima actualizada	Los endpoints o componentes nuevos tienen documentación técnica básica actualizada en el repositorio (README, comentarios de API, o documento técnico correspondiente).

Prácticas XP que hacen el DoD técnicamente verificable

Las prácticas de Extreme Programming seleccionadas para este proyecto convierten el DoD de una lista de intenciones en un conjunto de controles técnicos

verificables:

Test Driven Development (TDD)

El criterio 2 del DoD (“tests unitarios escritos y pasando”) se hace verificable porque TDD requiere que los tests se escriban antes del código de producción. Esto significa que al finalizar la HU, la existencia de tests no es opcional: es la evidencia de que el developer siguió el proceso correcto. Un developer no puede afirmar que una HU está “Done” sin tests, porque sin tests el código no pudo haberse escrito en ciclos TDD.

Integración Continua (CI)

El criterio 4 del DoD (“CI sin errores”) se hace objetivamente verificable porque el pipeline de CI es una verificación automática e imparcial. El equipo no puede argumentar que “funciona en mi máquina”: si el pipeline falla, la HU no está Done. El CI actúa como árbitro técnico neutral del DoD.

Pair Programming / Code Review

El criterio 3 del DoD (“Code Review completado”) se hace trazable porque la columna “Code Review” del tablero Kanban/Scrumban registra visualmente que la tarea pasó por revisión de pares antes de avanzar a la columna Review. Sin esa columna, el code review podría omitirse bajo presión de velocidad sin que el SM lo detecte.

Criterios de aceptación como contrato entre cliente y equipo

Los criterios de aceptación son el contrato funcional entre el cliente (PO que representa al usuario) y el equipo técnico. Cada criterio de aceptación responde a la pregunta: “¿Cómo sabemos que esta funcionalidad está correcta desde la perspectiva del usuario?”

Para este proyecto, los criterios de aceptación son especialmente relevantes en:

- EP03 (Catálogo de carreras): el cliente Admisiones (S3) debe validar que el catálogo refleja el pensum 2026 oficial. Los criterios de aceptación de EP03 incluyen la firma de validación de S3 en el hito H4.
- EP04 (Motor de recomendación): el PO debe definir qué combinación de respuestas de la encuesta vocacional produce qué resultado de recomendación. Los criterios de aceptación son las reglas del motor.
- EP06 (Panel de resultados): los criterios de aceptación incluyen que la interfaz sea comprensible para un aspirante de diversificado sin instrucción técnica (criterio de usabilidad verificado en H6).

Anti-patrón evitado: “La calidad no es solo visual”

Un error frecuente en proyectos de software es considerar que la calidad se mide únicamente por la apariencia de la interfaz o por la ausencia de bugs visibles en la demo. Este anti-patrón produce software que “funciona en la demo” pero que tiene deuda técnica acumulada, tests inexistentes, código no revisado y vulnerabilidades de seguridad no detectadas.

El DoD de este proyecto está diseñado para evitar este anti-patrón: una HU no está Done solo porque “se ve bien en el Sprint Review”. Está Done cuando el código tiene tests, el CI pasa, el code review fue completado y los criterios de aceptación verificados.

La calidad es un resultado del proceso, no un juicio visual al final del sprint.

Métricas de calidad cuantificables

La calidad en este proyecto no se mide únicamente por el cumplimiento del DoD. Se definen métricas cuantificables que el equipo debe alcanzar al cierre del proyecto:

Métrica	Valor mínimo aceptable	Cómo se mide	Responsable
Cobertura de tests unitarios	$\geq 70\%$ del código de producción en EP01–EP06	Reporte de cobertura del pipeline CI al cierre del sprint 8	Developer Chen
Tasa de defectos críticos en producción	0 defectos P1 en el MVP entregado en H7	Registro de bugs del período de staging (sprints 7–8)	SM Cordero
Criterios de aceptación aprobados sin retrabajo	$\geq 90\%$ de las HU aprobadas en Sprint Review sin volver al backlog por defectos de calidad	Velocidad efectiva vs velocidad bruta por sprint	PO Say
Puntaje de usabilidad en H6	$\geq 70\%$ de los participantes completan el flujo EP01–EP06 sin asistencia	Resultados de sesiones de usabilidad moderadas en sprint 8	PO Say + S8
Code Review completado	100% de las HU pasaron por la columna Code Review antes de Review del PO	Tablero Kanban/Scrumban — auditoría al cierre	SM Cordero

Aseguramiento vs control de calidad

El plan de calidad de este proyecto distingue dos actividades complementarias:

Aseguramiento de calidad (QA — proactivo)

El aseguramiento de calidad son las acciones preventivas que el equipo toma para garantizar que el proceso de desarrollo produzca calidad. En este proyecto, el QA incluye:

- El DoR como filtro de entrada: impide que historias mal definidas entren al sprint.
- Las sesiones de refinement del backlog: garantizan que las HU lleguen al Planning con criterios de aceptación claros.
- La práctica de TDD: garantiza que el código sea testeable por diseño, no por inspección posterior.
- El Pair Programming en módulos críticos (EP01, EP04): distribuye el conocimiento y reduce el riesgo de que un solo developer sea el único que entiende un componente.

Control de calidad (QC — reactivo)

El control de calidad son las verificaciones que el equipo realiza para detectar defectos en los entregables ya producidos. En este proyecto, el QC incluye:

- El pipeline de CI: detecta fallos de compilación y tests en cada commit.
- La columna Code Review: detecta deuda técnica, violaciones de estándares y pruebas insuficientes antes del merge.
- Las pruebas funcionales contra criterios de aceptación: detectan discrepancias entre lo construido y lo acordado con el PO.
- La revisión de seguridad por Developer Chen (EP01, EP02, EP06): detecta vulnerabilidades específicas antes de que lleguen a producción.

La diferencia entre QA y QC es que el QA evita crear defectos y el QC detecta defectos ya creados. Un proyecto con solo QC gasta recursos corrigiendo; un proyecto con QA + QC gasta menos en corrección porque produce menos defectos desde el inicio.